

%xmm3”的功能相当于 AVX 指令 “vsubss (%rdx), %xmm3, %xmm3” 的功能，都是将 XMM3 中低 32 位表示的单精度浮点数减去 RDX 指向的 4B 长度存储单元中的单精度浮点数，结果存放在 XMM3 寄存器中。

除了上述部分浮点处理指令以外，还有位操作和浮点数大小比较等指令，详细内容可参考 Intel 相关指令系统手册。

3.4 本章小结

任何一个 C 语言程序都要转换为对应机器所采用的指令集体系结构规定的机器代码才能执行。本章主要介绍 IA-32/x86-64 指令集体系结构的基础内容，包括 IA-32/x86-64 支持的数据类型、寄存器组织、寻址方式、常用指令类型、指令格式和指令的功能，从而为下一章介绍 C 语言程序在 IA-32/x86-64 架构上的机器级表示奠定基础。

习题

1. 给出以下概念的解释说明。

机器语言程序	机器指令	汇编语言	汇编指令
汇编语言程序	汇编助记符	汇编程序	反汇编程序
机器级代码	通用寄存器	变址寄存器	基址寄存器
栈指针寄存器	指令指针寄存器	标志寄存器	标志位 (条件码)
寻址方式	立即寻址	寄存器寻址	存储器操作数
相对寻址	基址寻址	变址寻址	实地址模式
保护模式	有效地址	比例变址	非比例变址
比例系数 (比例因子)	MMX 指令	SSE 指令集	SIMD
AVX 指令集	多媒体扩展寄存器	数据级并行处理指令	

2. 简单回答下列问题。

- (1) 一条机器指令通常由哪些字段组成？
- (2) 将一个高级语言源程序转换成计算机能直接执行的机器代码通常需要哪几个步骤？
- (3) IA-32/x86-64 中的逻辑运算指令如何生成标志位信息？移位指令可能会改变哪些标志位？
- (4) 执行条件跳转指令时所用到的标志信息从何而来？请举例说明。
- (5) 无条件跳转指令和调用指令的相同点和不同点是什么？

③ 对于以下 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) mov 8(%ebp, %ebx, 4), %eax
- (2) mov %al, 12(%ebp)
- (3) add (, %ebx, 4), %eax
- (4) or (%ebx), %dh
- (5) push %rcx

- (6) mov \$0xFFFF0, %eax
 (7) test %rax, %rax
 (8) lea 8(%ebx, %esi), %eax

④ 使用汇编器处理以下 IA-32/x86-64 中 AT&T 格式代码时都会产生错误, 请说明每一行代码存在什么错误。

- (1) movl 0xFF, (%eax)
 (2) movb %ax, 12(%ebp)
 (3) addl %ecx, \$0xF0
 (4) orw \$0xFFFF0, (%ebx)
 (5) addb \$0xF8, (%dl)
 (6) movl %bx, %eax
 (7) andl %esi, %esx
 (8) movq 8(%ebp, , 4), %rax
 (9) leaq 20(%rdi, %rsi), %eax

⑤ 假设变量 x 和 ptr 的类型声明如下:

```
src_type x;  
dst_type *ptr;
```

这里, src_type 和 dst_type 是用 typedef 声明的数据类型。有以下 C 语言赋值语句:

```
*ptr=(dst_type) x;
```

若 x 在寄存器 EAX 或 AX 或 AL 中, ptr 在寄存器 EDX 中, 则对于表 3.11 中给出的 src_type 和 dst_type 类型组合, 写出实现上述赋值语句的 IA-32 机器级代码, 要求用 AT&T 格式汇编指令表示。

表 3.11 题 5 表

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

⑥ 假设变量 x 和 y 分别存放在寄存器 RAX 和 RCX 中, 给出以下各指令执行后寄存器 RDX 中的结果。

- (1) leaq (%rax), %rdx
 (2) leaq 4(%rax, %rcx), %rdx
 (3) leaq (%rax, %rcx, 8), %rdx
 (4) leaq 16(%rcx, %rax, 2), %rdx
 (5) leaq (, %rax, 4), %rdx
 (6) leaq (%rax, %rcx), %rdx

⑦ 假设在 IA-32 系统中以下地址以及寄存器中存放的机器数如表 3.12 所示。

表 3.12 题 7 表

地址	机器数	寄存器	机器数
0x0804 9300	0xffff fff0	EAX	0x0804 9300
0x0804 9400	0x8000 0008	EBX	0x0000 0100
0x0804 9384	0x80f7 ff00	ECX	0x0000 0010
0x0804 9380	0x908f 12a8	EDX	0x0000 0080

分别说明执行以下指令后, 哪些存储单元地址或寄存器中的内容会发生改变? 改变后的内容是什么?

么？条件标志 OF、SF、ZF 和 CF 会发生什么改变？

- (1) `addl (%eax), %edx`
- (2) `subl (%eax, %ebx), %ecx`
- (3) `orw 4(%eax, %ecx, 8), %bx`
- (4) `testb $0x80, %dl`
- (5) `imull $32, (%eax, %edx), %ecx`
- (6) `mulw %bx`
- (7) `decw %cx`

⑧ 已知 IA-32 采用小端方式，根据给出的 IA-32 代码反汇编结果（部分信息用 x...x 表示）回答问题。

- (1) 已知 `je` 指令的操作码为 0111 0100，`je` 指令的跳转目标地址是什么？`call` 指令中的跳转目标地址 0x80483b1 是如何反汇编出来的？

```
804838c: 74 08          je      xxxxxxxx
804838e: e8 1e 00 00 00 call    0x80483b1<test>
```

- (2) 已知 `jb` 指令的操作码为 0111 0010，`jb` 指令的跳转目标地址是什么？`movl` 指令中的目的地址是如何反汇编出来的？

```
8048390: 72 f6          jb      xxxxxxxx
8048392: c6 05 00 a8 04 08 01 movl    $0x1, 0x804a800
8048399: 00 00 00
```

- (3) 已知 `jle` 指令的操作码为 0111 1110，`jle` 和 `mov` 指令的地址各是什么？

```
xxxxxxx: 7e 16          jle     0x80492e0
xxxxxxx: 89 d0          mov     %edx, %eax
```

- (4) 已知 `jmp` 指令的跳转目标地址采用相对寻址方式，`jmp` 指令操作码为 1110 1001，其跳转目标地址是什么？

```
8048296: e9 00 ff ff ff jmp     xxxxxxxx
804829b: 29 c2          sub     %eax, %edx
```

⑨ 对于以下 x86-64 中的 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后缀，并说明每个操作数的寻址方式。

- (1) `mov 8(%rbp, %rbx, 4), %ax`
- (2) `mov %al, 12(%rbp)`
- (3) `add (%rbp, %rsi, 4), %r9w`
- (4) `or (%rbx), %dil`
- (5) `sub %bpl, 8(%rsp, %rdi, 4)`
- (6) `mov $0xFFFF0, %eax`
- (7) `test %r8d, %r8d`
- (8) `lea 8(%rbx, %rsi), %rax`
- (9) `xor $0xF0, %rax`

10. 假设 x86-64 系统某程序中变量 x 和 ptr 的类型声明如下:

```
src_type x;  
dst_type *ptr;
```

这里, src_type 和 dst_type 是用 typedef 声明的数据类型。有以下几个 C 语言赋值语句:

```
*ptr=(dst_type) x;
```

若 x 存储在寄存器 RAX 或 EAX 或 AX 或 AL 中, ptr 存储在寄存器 RDX 中, 则对于表 3.13 给出的 src_type 和 dst_type 的类型组合, 写出实现上述赋值语句对应的汇编指令。

表 3.13 题 10 表

src_type	dst_type	汇编指令 (AT&T 格式)
char	long	
int	char	
int	unsigned long	
short	int	
unsigned char	unsigned	
char	unsigned long	
unsigned long	int	